

## Phase 3 Report

### Testing

In phase 3, we implemented automated unit and integration testing, with the aim of testing everything we possibly could and emphasizing code coverage. Testing was conducted using the JUnit framework alongside Mockito. By the end, we reached an average of 64% line coverage, and 71% branch coverage among all of our packages. Below is a table highlighting the numbers we were able to achieve:

| Package     | Line Coverage | Branch Coverage |
|-------------|---------------|-----------------|
| Screens     | 37%           | 50%             |
| Game        | 68%           | 67%             |
| Entities    | 84%           | 80%             |
| Objects     | 63%           | 100%            |
| Asset       | 34%           | 33%             |
| Pathfinding | 99%           | 89%             |
| Components  | 100%          | 85%             |

### Unit Testing Features

We designed unit tests to validate single features in strict isolation, stripping away external dependencies. Here, we focused on verifying the logic of our classes without relying on the broader game loop.

**Player:** PlayerTest verified the player entity's core systems, including health, taking damage, and ensuring excess healing converts into score. Time-based mechanics were also isolated, successfully validating maximum speed clamping, immunity timer decay, stun duration logic, and the graphite drain mechanic (losing health over time while moving).

**Objects (WhiteOut, Pickup, Ink, Door):** Tests across various interactive entities validated their logic, this includes things such as initialization scaling, hitbox generation, and player collision. Using Mockito, a Java based mocking framework, it allowed us to create simulated "dummy" versions of complex external dependencies, which in this case was the Player entity. It allowed us to test Pickup entities; Mockito's verification methods ensured that Pickup merely attempted to call the player's healing method (and then correctly flag itself for destruction), which is what we care about in an isolated unit test. The same isolated approach was applied to our hazards. We confirmed WhiteOut applies accurate damage and respects cooldown timers to prevent multi-hit

frames. Ink correctly applies movement multipliers. Finally, static objects like Door were verified to initialize physical bounds correctly based on their passed string identifiers.

**Enemy (MobileEnemy):** MobileEnemy's testing was aimed at correctly managing the enemy's condition, switching Chase and Patrol modes depending on the distance to the player, building the correct trajectory when necessary, as well as maintaining the current trajectory after the timer expires; this was done without relying on the actual A\* pathfinding or physical map files. The conditions for following the route and stopping the enemy when the goal is reached or the player is absent were checked.

**Asset:** In testing classes within our asset package, which are designed mainly as a delegation wrapper for LibGDX's native AssetManager, we decided to omit large portions of code. We maintained our testing focus on game logic, hence we were able to achieve maximum branch and line coverage in our MapAsset enum. This tested the simple logic in the selection of levels, which we found to be more relevant.

**Pathfinding (AStar):** AStar testing was based on route generation checks on the tile grid. These tests confirm that the algorithm works successfully on both open maps and maps with blocked cells.

**Pathfinding (RandomPatrol):** RandomPatrol testing was based on the validation of an algorithm for finding random acceptable places to move an enemy to a small distance if the player is not detected. Borderline cases have been checked where the enemy must stay within the map and avoid blocked cells when making a route. These checks ensure that collision and navigation rules are followed.

**ScreenManager:** All screen interactions were tested with a "Test Mode" boolean defined in GdxGame. While this boolean is active, any file-related operations, such as data read/write and loading textures are bypassed, therefore isolating the core structure of the interactions. As a result, multiple classes were refactored to centralize their external file calls, and incorporate proper null-checking where possible. Testing the ScreenManager confirmed a proper singleton design, and confirmed that the result of calling a screen setting function matched the game's screen.

**Menu Screens:** Unit testing the Main Menu primarily focused on two points of functionality: layout switches and button interactions. Each layout was individually tested to ensure that they can be displayed, and that the changeLayout method assigns each one correctly. Each button was ensured to display the correct toggle data, and trigger the correct method between layout changes, playing a specific level, changing settings, and exiting the program. Similarly, in GameScreenTest, the layout switches and button interactions were tested to near identical results.

Additionally, the pause/unpause functionality was confirmed through the toggled boolean to freeze the game state.

**Game World:** Unit testing for the game world primarily aimed to validate the correctness of its update loop, floor drawing system, and its collision handling. The update loop was tested to ensure that all entities spawned within the world were properly handled and disposed of using dummy entities, and that pickups properly spawned within the world. Collision handling was tested in bulk, ensuring that entities would end their movement beside any blocking object moving in any direction, world borders included.

**Components:** Unit testing for Components mostly aimed towards validating the correctness of the setters, and utility functions within the Transform and Vec2 Classes.

**Audio:** AudioManager unit tests included testing the on/off toggles, the music controls, the sound loops. The music track was verified to abide by the toggle in settings, and additionally sets to half volume when a menu is open. The move and stunned sound loops were tested through various conditions, such as when inside an ink puddle, if sfx is toggled off, and if it's toggled off during the sound loop.

## Integration Testing Features

Our integration testing was designed to verify the interactions between multiple game systems. The focus was to ensure communication functioned correctly within the broader game loop.

**World Loading:** WorldLoaderTest verifies the 2D array grid logic and collision boundary math. Importantly, it also verifies the Tiled map property parser and JSON deserialization. In other words, this means it can read the "entities" layer from the map as well as JSON files, meaning our two methods of entity spawning are working as expected. We intentionally omitted branch coverage for the remaining types of entity spawning conditions, as they all share identical instantiation logic, and testing one branch sufficiently verified the system. This can explain the lower branch coverage for our Game package.

**Game World:** Within the game world unit testing, specifically the update loop, the main entities within the game are tested in their collisions with the player within a controlled setting. For the Eraser class, it was tested that upon collision that the player was damaged. The sharpener was tested to apply the stunned effect to the player. The collectibles were tested for their respective effects, and that they modified the points.

**Screens:** In the main menu, the PlayerData integration was verified to present the correct level options to the player and display the high score for each level, initialized at 0 if not completed. This was tested in both the label of each level select button, and ensuring screen changes to the

correct next level based on the highest level completed. The PlayerData integration within the game screen was unique, by verifying correct writing of data to the PlayerData object upon level completion. The high score was tested to overwrite the old one if it surpasses the highest, and the highest level available value is updated upon completing a new level.

**Dynamic Enemies (Eraser & Pencil Sharpener):** Our tests validate the communication between the enemy's attack mechanics and the player's health state. Furthermore, the tests for our dynamic enemies also focused on determining the behavior of opponents in combat in different states. The tests allowed us to confirm that the starting position is being correctly saved and used as a reset point after a confrontation with the player. The attack cooldown was also checked, that there is no duplication of attacks and the enemy is waiting for the cooldown timer for the next attack. Additional tests were also performed on the movement and correct orientation of the sprite, depending on the movement.

**Objects (ExitPoint):** ExitPoint was verified to successfully trigger the global game win condition upon intersection.

## Test Quality and Coverage

**Coverage:** Many tests were written explicitly to satisfy complex branch conditions, ensuring high branch coverage. For instance, we can look at our WorldLoaderTest; all conditions ranging from malformed entities to null tiles were thoroughly tested to ensure the loader handles them safely without crashing. In addition, entity tests verified both the true and false evaluation branches of invulnerability and cooldown timers.

In implementing these tests, heavy analysis was committed on branch coverage reports. They provided a deep dive into how the Java compiler evaluates logical operators (&& and ||). We learned to meticulously design test vectors that specifically trigger the second operand of these statements, forcing the evaluation of edge cases that would otherwise remain hidden. This process is what helped us in achieving high branch coverage in many of our classes.

## Code Omissions

When it comes to testing classes with heavy use of textures and external files, such as screen, audio, or data classes, there is a significant reduction in the code coverage metric because we intentionally omitted physical asset loading. Our unit tests cause framework crashes when attempting to load native graphics, so we created a workaround that allows for null textures while testing. Any line or branch that involves creating or loading a new texture is omitted from the test suite, meaning a lower coverage metric is produced. This lower metric does not indicate lack of test quality, since all omitted code is easily tested manually by ensuring all textures specified appear within the game at the proper location.

Similarly, we also chose to skip testing physical keystrokes (`Gdx.input`), as it triggers a state that relies entirely on Mockito faking the keyboard. This tests the framework rather than the game logic. Instead, we focus on testing relevant concepts within the game itself. For example, we tested the mathematical boundaries of the movement engine (like proving contradictory left/right inputs mathematically cancel out to zero, or that acceleration is safely clamped by speed limits). Testing inputs seemed especially trivial when one could easily examine its functionality within the game environment.

Furthermore, we omitted the `create()`, `render()`, and `dispose()` methods. Testing these methods would require us to attempt to mock the entire GPU pipeline, which would yield unnecessary and trivial tests. Instead, we restricted our testing in these areas to simply internal variables. To make sense of this, a tangible example of this is verifying the true/false booleans for our Audio Settings update correctly when toggled. Instead of trying to test the actual speaker hardware, this was more valuable for us.

With these reasons, it explains why we were unable to achieve maximum coverage for many of our packages. We decided that it would be wiser to focus our tests where it actually made a difference.

## Bugs Revealed

**Issue:** The floor drawing function did not check for validity that the calculated weight lied within the acceptable range, leading to the possibility of the renderer attempting to draw with opacity above 100% or in the negatives.

**Fix:** Before applying the brush directly to the tilemap, the weight is first checked and clamped into its acceptable range.

**Issue:** The death-check condition (`health <= 0`) was originally coupled to the excess-healing check via an else if statement. This design meant that if a player was healed past their maximum health, the program skipped the death evaluation entirely for that frame.

**Fix:** The logic was refactored into independent and sequentially evaluated execution blocks to guarantee that game-over states are consistently validated. This should occur regardless of prior healing.

**Issue:** Time-based simulation tests revealed that the `stunTimer` was decrementing indefinitely into negative values during the `update(delta)` loop, rather than cleanly terminating the status effect.

**Fix:** The update loop was patched to explicitly clamp the timer at 0.0f upon completion, preventing unpredictable mathematical behavior in dependent game state logic.

**Issue:** The player continued to play motion sounds and consume graphite when pressing the motion key into the wall, although no movement actually occurred.

**Fix:** The motion logic has been changed, so that now the sound effects of motion and graphite consumption are guided by a real displacement in space.

**Issue:** In some parts of the map, when the player pressed against the wall, the enemies lost the player because they found themselves in a blocked cell. This was because of the partial overlap of the tile with obstacles placed on top.

**Fix:** The attack logic has been changed so that enemies can attack from a short distance away from the player and not just through direct contact. This fixed the situation where the player could be pinned against a wall and be out of bounds for enemies.

## Refactoring

To ensure high-quality, maintainable test code, we strictly adhered to D.R.Y. (Don't Repeat Yourself) principles. Firstly, we prioritized inheritance; we created an abstract `GameTest` base class. This centralizes the heavy Mockito setup required for LibGDX environments (mocking OpenGL, file systems, and singletons), preventing memory leaks and removing hundreds of lines of redundant copy paste code from individual entity tests. More importantly, it drastically improved our scalability.

In order to test core functionalities without loading each asset, much of the code was refactored to accommodate a testmode, and operate identically regardless of textures, sounds, or save files being loaded. A `testMode` boolean was added to `GdxGame`, and many of the inheriting classes utilize this boolean to omit the unnecessary asset-dependent branches while running tests.

In addition, writing assertions exposed a weakness in our architecture where tests were tightly coupled to arbitrary game design values (e.g., raw hitbox dimensions or damage integers). We refactored to replace these hardcoded values with public static final constants. Tests now assert against these constants, instead of a hardcoded number that is not easily changeable without breaking our test suite.